

MANUAL26. УРОК 4. ВИДЫ ТЕСТИРОВАНИЯ: КАК ВЫБРАТЬ ПРОВЕРКУ ПОД РИСК.

На прошлом уроке вы рассматривали SDLC и учились видеть, на каком этапе жизненного цикла продукта проблему можно поймать раньше и дешевле. Это ключевой навык для начинающего QA: он помогает понимать не только «что сломалось», но и «почему это вообще дошло до пользователя». В этой теме мы идем на шаг глубже. Теперь важно не просто заметить риск, а выбрать правильный способ проверки. Именно этому и посвящен урок: как из риска прийти к осмысленному набору тестов.

Что мы разбираем в этом уроке

Главная цель темы — научиться различать функциональные и

нефункциональные проверки не «по определению из учебника», а в формате реального рабочего решения. Мы разберем, какие вопросы закрывает каждый вид тестирования, почему оба вида одинаково важны для качества, как они связаны между собой в одном релизе и как не потерять фокус, когда времени мало. В конце урока у вас должен остаться не список терминов, а рабочая логика: риск -> тип проверки -> аргумент для команды.

Почему в QA вообще есть разные виды тестирования?

Когда начинаешь в QA, легко попасть в ловушку: «нажал кнопку, результат есть, значит все хорошо». На короткой демо это часто выглядит правдой. Но в реальном релизе пользователь оценивает не только сам факт работы функции, а весь опыт целиком: быстро ли откликается экран, не зависает ли игра, не ломается ли поток действий.

Именно поэтому в тестировании есть разные виды проверок. Это не лишняя теория, а практичный способ смотреть на продукт с разных сторон. Один тип

проверяет логику: система
делает то, что должна. Второй
проверяет качество исполнения:
удобно ли, стабильно ли,
предсказуемо ли это работает у
реальных пользователей.

Хороший пример из игры: новая
механика наград на демо
выглядит корректно, игрок
получает бонус. Но в проде у
части аудитории экран
подвисает, награда появляется с
задержкой, а на слабых
устройствах падает FPS.
Формально функция «есть», но
пользователь остается
недоволен. Поэтому главный
принцип урока простой: вопрос
«работает/не работает» слишком
узкий. В QA важно задавать
второй вопрос: «как именно это
работает для человека в
реальных условиях?».

Что такое функциональное тестирование?

Функциональное тестирование
отвечает на вопрос: выполняет
ли система нужное поведение по
требованиям и сценариям
пользователя. Это проверка
корректности логики. Проще
говоря, мы сравниваем
договоренность «как должно

быть» и фактический результат «что система сделала на самом деле».

Здесь фокус всегда на цели пользователя. Если игрок должен войти в событие и получить награду, то мы проверяем путь от входа до результата, а не отдельную кнопку вне контекста. Если игрок должен сохранить прогресс, мы проверяем не один экран, а целостный поток с выходом, перезаходом и повторной проверкой данных. Функциональное тестирование требует смотреть на продукт сценариями, иначе легко получить красивый, но бесполезный результат вида «элемент работает, а путь сломан».

У функциональных проверок есть важная привычка, которую полезно развивать с раннего уровня: задавать уточняющие вопросы к требованиям. Что считается успешным выполнением сценария? Где крайние условия? Что должно происходить при ошибке? Как система себя ведет, если пользователь отменил действие на середине? Эти вопросы не «придирки», а нормальная часть QA-мышления. Чем четче ответы, тем точнее проверка.

А что такое нефункциональное тестирование?

Нефункциональное тестирование отвечает на другой вопрос: как система выполняет уже правильную логику. Это проверка качества исполнения. Сюда относятся характеристики, которые пользователь чувствует напрямую: скорость, стабильность, удобство, совместимость, читаемость интерфейса, предсказуемость поведения.

Важно отдельно проговорить: нефункциональные проверки не являются «дополнением на случай свободного времени». В играх и приложениях с высокой конкуренцией именно эти аспекты часто решают, останется ли человек в продукте. Игрок может простить редкую неточность, но он почти всегда замечает длинную загрузку, фризы и падения. Поэтому команда, которая откладывает нефункциональные проверки «на потом», обычно переносит в релиз самые болезненные риски.

Еще один сильный эффект нефункционального тестирования — переход от общих слов к измеримым

формулировкам. Вместо фразы «работает быстро» появляется понятный ориентир на время отклика в заданных условиях. Вместо «стабильно» — наблюдаемое отсутствие критических падений на целевых платформах в типичном сценарии сессии. Именно такие формулировки делают обсуждение с командой предметным.

Почему эти два вида нельзя разрывать?

На практике функциональные и нефункциональные проверки работают как две стороны одной задачи. Если оставить только функциональную часть, можно получить продукт с правильной логикой, но слабым пользовательским опытом. Если оставить только нефункциональную часть, можно получить «гладкую» систему, которая быстро работает, но решает задачу неправильно. В обоих случаях итог один: пользователь недоволен, команда тушит пожары.

Хороший QA не спорит, что важнее. Он связывает оба уровня в единый набор проверок и объясняет, почему именно

такой баланс нужен для текущего релиза. Это особенно важно в условиях ограниченного времени. Ограничение по времени не отменяет качество, оно требует приоритезации. Сильный тестировщик в такой ситуации не пытается «проверить все», а выбирает то, что надежнее всего защищает критичный путь игрока.

Практический взгляд команды: как виды тестирования влияют на решения

Внутри команды различие между функциональными и нефункциональными проверками помогает не только в тестировании, но и в планировании релиза. Когда QA формулирует риск, команда быстрее понимает, о чем речь: проблема в логике функции или в качестве ее работы. Это сокращает количество «пустых» обсуждений и ускоряет принятие решений.

Например, если проблема в логике начисления награды, обсуждение идет вокруг правил, условий и корректности переходов между состояниями.

Если проблема в производительности экрана наград, разговор уже про нагрузку, рендер, поведение на разных устройствах и приоритет исправления перед релизом. Внешне это один и тот же функциональный блок, но тип проверки меняет фокус и план действий.

Еще один важный момент: QA-аргументация становится сильнее. Вместо абстрактного «надо больше тестировать» появляется предметное объяснение: какой риск, какой вид проверки его закрывает и что произойдет, если проверку убрать. Такой стиль коммуникации быстро повышает доверие к работе тестировщика.

Карта типов тестирования

Карта типов тестирования — это компактная рабочая схема, которая помогает быстро перейти от риска к набору проверок. Это не список «всех тестов мира» и не бюрократический документ ради документа. Ее задача практическая: удержать фокус команды на главном и не

потерять критичные проверки
под давлением сроков.

Когда времени мало, люди
склонны действовать хаотично:
проверяют то, что быстрее
проверить, а не то, что важнее
для пользователя. Карта типов
снижает этот риск. Она помогает
сначала посмотреть на проблему
через два фильтра —
корректность логики и качество
исполнения, — а уже потом
добавлять конкретные проверки.

В базовом виде такая карта
обычно строится вокруг
нескольких устойчивых
направлений. Для
функциональной части это
позитивный поток, негативные
сценарии и граничные условия.
Для нефункциональной части —
стабильность,
производительность,
совместимость и удобство. Здесь
важно понимать, что это
стартовая рамка, а не строгий
шаблон. Под разные задачи
акценты меняются.

Если релиз затрагивает
авторизацию, в функциональной
части вы проходите правильный
вход, ошибки при неверных
данных и сценарии неполного
действия. В нефункциональной
части смотрите, не выросло ли
время входа, нет ли

нестабильности на переходах и одинаково ли ведет себя сценарий на целевых платформах. Такой подход дает не «идеальную полноту», а реалистичное и честное покрытие рисков.

Как не превратить карту в формальность

Карта полезна только тогда, когда ее связывают с конкретным пользовательским риском. Если писать в ней общие фразы вроде «проверить стабильность» без контекста, она быстро становится декоративной. Рабочая формулировка всегда отвечает на три вопроса: что может пойти не так, где это проявится у пользователя и какой проверкой мы это ловим.

Полезно регулярно задавать себе вопрос: если убрать эту проверку, что именно может произойти после релиза? Если ответа нет, значит проверка либо слишком абстрактна, либо сейчас не приоритетна. Такой короткий фильтр помогает держать карту живой и практичной.

Как QA выбирает вид проверки под риск

В реальной задаче помогает простой маршрут принятия решения. Сначала риск формулируется максимально предметно: что именно игрок не сможет сделать, где он потеряет прогресс или в каком месте получит негативный опыт. Затем риск разделяется по природе: это проблема логики, проблема качества исполнения или смешанный вариант. После этого выбирается минимально достаточный набор проверок, который закрывает обе стороны.

Дальше идет приоритизация. Если сценарий относится к критичному пути, ему отдается приоритет в первую очередь. Критичный путь — это шаги, без которых пользователь не достигает основной цели. В игровом контексте это может быть вход в матч, начисление награды, сохранение прогресса, работа ключевых экранов в боевом режиме. Пропуск проверки на критичном пути обычно дороже любой «мелкой» экономии времени.

Отдельно важен шаг, который часто пропускают: фиксация ограничений. Команда должна

честно зафиксировать, что именно в это релизное окно проверить не успели и какой остаточный риск принимается. Это зрелая практика. Иллюзия «мы проверили все» почти всегда приводит к неприятным сюрпризам.

Такой подход меняет и формат общения с командой. QA приносит не просто набор тест-кейсов, а решение: какой риск видим, каким типом проверки закрываем и почему именно так. Это делает вклад тестировщика прозрачным и ценным для релизного решения.

Мини-разбор: одинаковая проблема, разный тип проверки

Полезно увидеть это на коротком примере. Допустим, игрок жалуется: «после завершения события награда не отображается». Сначала кажется, что это одна проблема, но у нее может быть разная природа.

1. **Первый вариант:** награда действительно не начисляется. Это функциональный дефект,

потому что ломается логика сценария.

2. **Второй вариант:** награда начислилась, но экран подвис и не обновился вовремя. Это нефункциональный дефект, связанный с качеством отображения и отклика интерфейса.
3. **Третий вариант:** награда начисляется только в части условий, а в остальных потоках срабатывает нестабильно. Это уже смешанный случай, где нужны оба типа проверок.

Именно поэтому QA важно не останавливаться на симптоме, а докапываться до природы проблемы. От этого зависит, какие проверки будут полезны, а какие окажутся шумом.

Попробуем разобрать тему на игре Fortnite

Чтобы теория не оставалась абстрактной, берем практический контекст Fortnite. В продукт добавляют новый временный режим события: отдельная очередь входа, система наград и обновленный интерфейс матча. На уровне фичи это выглядит как «одно

изменение», но для QA это сразу набор разнотипных рисков.

Функциональный слой проверок здесь отвечает за логику сценария. Игрок должен корректно попасть в режим, пройти событие, получить награду по правилам и сохранить результат после выхода и повторного входа. Если где-то в этой цепочке есть разрыв, пользователь теряет цель и доверие к системе.

Нефункциональный слой проверок смотрит, как эта же логика ведет себя в реальных условиях. Насколько стабильно работает клиент в напряженной игровой сессии, укладывается ли вход в матч в приемлемое время, одинаково ли понятен интерфейс на целевых разрешениях, не мешают ли визуальные элементы ориентироваться в боевой динамике. Эти вопросы не менее важны, чем логика, потому что именно они определяют ощущение «игра комфортна» или «игра раздражает».

Хорошо видно, что один и тот же пользовательский путь требует сразу двух углов проверки. Если оставить только функциональную часть, можно выпустить корректную, но «тяжелую» и нестабильную

фичу. Если оставить только нефункциональную часть, можно выпустить гладкий интерфейс со сломанной логикой наград. В обоих случаях релиз будет проблемным, просто проблемы проявятся по-разному.

Что делает QA на разборе результатов

После прохождения проверок важна не только фиксация «прошло/не прошло», но и качественный разбор картины риска. Сильный разбор обычно отвечает на четыре вопроса: какие сценарии подтверждены, где найдены отклонения, какой риск для игрока остается и что команда делает с этим риском до релиза.

Такой формат полезен тем, что переводит тестирование из «набора наблюдений» в язык решений. Команда понимает, где можно выпускаться безопасно, где нужен фикс до релиза, а где допустим выпуск с контролируемым ограничением и планом мониторинга.

Типичные ошибки в этой теме и как их

избежать

1. Первая частая ошибка — проверять отдельные элементы вместо полного сценария. В таком режиме можно получить десятки «зеленых» проверок и все равно выпустить поломанный пользовательский путь. Исправление простое, но дисциплинарное: тестировать цепочку до конечной цели игрока.
2. Вторая ошибка — откладывать нефункциональные проверки «на потом». Обычно именно это «потом» не наступает, и проблемы стабильности выходят уже после релиза. Лучше заранее заложить хотя бы минимальный обязательный блок нефункциональных проверок для критичного пути.
3. Третья ошибка — пытаться покрыть все, не расставив приоритеты. Это выглядит как активная работа, но часто приводит к распылению времени. Рабочий подход другой: сначала риск, потом тип проверки, потом приоритет, и только после этого — детализация кейсов.
4. Четвертая ошибка — не фиксировать остаточный риск. Когда команда не

проговаривает, что именно осталось непроверенным, она теряет прозрачность и готовность к последствиям. Фиксация остаточного риска — это не признание слабости, а признак профессиональной зрелости.

5. Пятая ошибка — использовать слишком общие формулировки в отчетах. Фразы вроде «проверили стабильность» почти бесполезны без контекста. Гораздо полезнее указывать, какой сценарий, в каких условиях, какой результат и какой риск для пользователя.

Итоги занятия

В этом уроке мы сделали важный переход от «тестирования как списка действий» к «тестированию как выбору стратегии». Вы увидели, что функциональные и нефункциональные проверки не конкурируют между собой, а работают в связке. Функциональные проверки отвечают за корректность логики. Нефункциональные показывают, насколько надежно и удобно эта логика работает в реальной жизни пользователя.

Если коротко, профессиональный смысл урока такой: QA ценен не количеством проверок, а качеством выбора. Сильный тестировщик умеет связать риск с подходящим видом тестирования, объяснить этот выбор команде и честно зафиксировать оставшиеся ограничения перед релизом.

Глоссарий к занятию

1. **Функциональное**

тестирование – это проверка того, что система корректно выполняет требуемые функции и пользовательские сценарии.

2. **Нефункциональное**

тестирование – это проверка качественных характеристик системы: скорости, стабильности, совместимости, удобства и других параметров исполнения.

3. **Критичный путь**

пользователя – это последовательность шагов, без которой пользователь не может достичь основной цели в продукте.

4. **Карта типов тестирования** –

это компактная рабочая схема, которая помогает выбрать нужные виды проверок под конкретный риск.

5. Остаточный риск – это риск, который остается после проведенных проверок и осознанно принимается командой на релизе.